OS Laboratory MSE — revision sheet (Experiments 1–10)

details and line-by-line explanations: see the full study guide

Basics used everywhere

- Exit status: `0` = success, non-zero = failure (`echo $?`). `perror("x")` prints `x: reason`.
- Read a word safely: `char a[100]; scanf("%99s", a);` (no `&` for arrays).
- Descriptors: **0** stdin (keyboard), **1** stdout, **2** stderr; `open` returns the lowest free number (3).
- Compile `gcc f.c -o f`; add `-pthread` for Exp. 9. Man pages: `man 2 fork`, `man 3 printf`.

Exp 1 — exec()

```
scanf("%99s", sname);
if (access(sname, R_OK) == -1) { perror("..."); return 1; }
execl("/bin/sh", "sh", sname, (char*)NULL);
perror("...");  return 1;     /* only if exec failed */
```

exec **replaces** the process (same PID); it returns only on failure (-1). `sh script` needs only *read* permission. Script (the three-number test):

```
read a b c
if [ "$a" -ge "$b" ] && [ "$a" -ge "$c" ]; then echo "$a is the greatest"
elif [ "$b" -ge "$a" ] && [ "$b" -ge "$c" ]; then echo "$b is the
      greatest"
else echo "$c is the greatest"; fi
```

The shebang is ignored by `sh script`; portable prompt = printf + read.

Exp 2 — read a file with system calls

```
fd = open(fname, O_RDONLY);  if (fd == -1) { perror("Error"); return 1; }
fflush(stdout);
while ((n = read(fd, buffer, sizeof(buffer))) > 0)
    write(STDOUT_FILENO, buffer, n);       /* n bytes, not sizeof */
close(fd);
```

`read` returns bytes read, **0 at end of file**, -1 on error. `ssize_t n` (signed). Extra parentheses around `n = read(...)`.

Exp 3 — five children with fork()

```
for (i = 1; i <= 5; i++) {
    fflush(stdout);
    pid = fork();
    if (pid < 0)        { perror("fork"); return 1; }
    else if (pid == 0)  { printf(.. getpid(), getppid()); exit(0); }
    else                  printf(".. child pid %d", pid);
}
for (i = 0; i < 5; i++) wait(NULL);
```

`fork` returns: child's PID in the parent, **0 in the child**, -1 on failure. The child must `exit(0)`; without it you get $2^5 = 32$ processes. Output order is *not* fixed. Zombie = ended child not yet waited for.

Exp 4 — write keyboard text to a file

```
fd = open(fname, O_CREAT | O_WRONLY | O_TRUNC, 0644);
while ((n = read(0, buffer, sizeof(buffer))) > 0)
    write(fd, buffer, n);
close(fd);
```

`O_CREAT` create, `O_WRONLY` write only, `O_TRUNC` erase old content. `0644` = `rw-r--r--` (octal; r=4, w=2, x=1), used only when the file is created. **Ctrl+D**: `read` returns 0 (press it twice if the line already has text).

Exp 5 — signals

```
void signalhandler(int sig) { if (sig == SIGINT) printf(".."); ...
    if (sig == SIGTERM) { printf(".."); exit(0); } }
signal(SIGINT, signalhandler);  /* also SIGTERM SIGHUP SIGALRM SIGUSR1 */
alarm(20);  printf("%d", getpid());
while (1) { printf("Waiting\n"); sleep(5); }
```

Linux numbers: HUP 1, INT 2 (Ctrl+C), KILL 9, USR1 10, ALRM 14, TERM 15, STOP 19. **SIGKILL and SIGSTOP cannot be caught.** `alarm` fires once. `sleep` returns early when a handler runs. Send with `kill -SIGINT PID` from a 2nd terminal. **printf/exit are not async-signal-safe; best: sigaction + write + flag.**

Exp 6–8 — scheduling formulas

- $\text{TAT} = \text{CT} - \text{AT}$, $\text{WT} = \text{TAT} - \text{BT}$ ($\text{AT} = 0$: $\text{TAT} = \text{CT}$).
- **FCFS:** `wt[0]=0; wt[i]=wt[i-1]+bt[i-1]; tat=wt+bt`. With arrivals: $\text{CT}_i = \max(\text{CT}_{i-1}, \text{AT}_i) + \text{BT}_i$.
- **SJF:** sort by burst (tie: smaller pid), swap *whole structs* via `temp`, then FCFS formulas. Optimal average WT; starvation; burst must be predicted ($\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$). SRTF = pre-emptive SJF.
- **RR:** each turn runs $\min(q, \text{remaining})$.

```
while (flag == 0) { flag = 1;
  for (i = 0; i < n; i++) if (rem[i] > 0) { flag = 0;
    if (rem[i] > tq) { time += tq; rem[i] -= tq; }
    else { time += rem[i]; rem[i] = 0; tt = time; wt = tt - bt; } } }
```

Averages: total/n with `double` or `(float)` cast (else integer division). Very large q = FCFS; very small q = many context switches. Check: bursts 3,4,2,5, $q = 2$ ⇒ avg WT 6.50, avg TAT 10.00.

Exp 9 — producer–consumer

```
sem_init(&empty,0,N);  sem_init(&full,0,0);  sem_init(&mutex,0,1);
producer: sem_wait(&empty); sem_wait(&mutex); put; in=(in+1)%N;
          sem_post(&mutex); sem_post(&full);
consumer: sem_wait(&full);  sem_wait(&mutex); get; out=(out+1)%N;
          sem_post(&mutex); sem_post(&empty);
```

The manual's program has no semaphore: `item` counts items; `item==N` means full, `item==0` means empty. To use real ones in the menu program: `sem_trywait(&empty)==-1` ⇒ "Buffer is full", then `sem_post(&full)`; consume is the mirror image. Swapping the two waits in the producer can **deadlock**. Compile with `-pthread`.

Exp 10 — Banker's algorithm (safety test)

```
need[i][j] = max[i][j] - alloc[i][j];  finish[i] = 0;
while (count < p) { found = 0;            /* reset in EVERY pass */
  for (i = 0; i < p; i++) if (finish[i] == 0) {
    for (j = 0; j < r; j++) if (need[i][j] > avail[j]) break;
    if (j == r) { for (k = 0; k < r; k++) avail[k] += alloc[i][k];
                  safeseq[count++] = i;  finish[i] = 1;  found = 1; } }
  if (found == 0) { puts("unsafe"); return 0; } }
```

Need = Max − Allocation. A process can run if Need ≤ Available; when it finishes it returns its Allocation. Safe state ⇒ no deadlock; unsafe ⇒ deadlock *possible*. Cost about $m\,n^2$. Request of P_i: Request ≤ Need, Request ≤ Available, pretend to grant, run the safety test, undo if unsafe. Notebook data ⇒ safe sequence P1 P3 P4 P5 P2. **Without found = 0 inside the while, an unsafe state can loop for ever.**

Return values to remember

`open`	fd, or -1
`read`	bytes; **0 = EOF**; -1 error
`write`	bytes written; -1
`fork`	PID / **0** / -1
`wait`	PID of ended child; -1 none left
`exec*`	**only returns on failure** (-1)
`access`	0 ok; -1 no
`alarm`	seconds left on old alarm
`scanf`	items read, or EOF

Viva one-liners

- **fork vs exec:** fork copies a process; exec replaces the program inside the same process.
- **Zombie / orphan:** ended child not yet waited for / child whose parent ended first.
- **System call vs library:** `open read write fork` enter the kernel; `printf scanf` are buffered library functions.
- **Signal:** asynchronous notification; options: default, ignore, handler.
- **Convoy effect:** short jobs wait behind a long one (FCFS).
- **Why SJF is optimal:** short job first lowers total waiting; but burst times are unknown.
- **Semaphore:** integer changed only by atomic wait (P) and signal (V).
- **Why 3 semaphores:** `empty`=N blocks producer, `full`=0 blocks consumer, `mutex`=1 excludes.
- **Critical section:** code touching shared data, one thread at a time.
- **Deadlock needs all four:** mutual exclusion, hold and wait, no pre-emption, circular wait.

Exam-day checklist

- Read the aim; write the includes that your calls need (`unistd.h`, `fcntl.h`, `sys/wait.h`, `signal.h`, `semaphore.h`).
- Check results of `open`, `fork`, `access`, `scanf` and print the reason with `perror`.
- Compile with `-Wall`; run your sample input and compare with the manual.
- Scheduling: say your assumption (all arrival times 0), print the table, totals and averages with `%.2f`.
- Exp 3 / 9: if the output order differs, explain that the scheduler decides.